

# GRAPH DISTILLATION: TRANSFERRING ATTRIBUTION-GRAPH STRUCTURE FROM A LARGE TEACHER TO A SMALL STUDENT

**Anonymous authors**

Paper under double-blind review

## ABSTRACT

*[Abstract. One paragraph: the gap between output-level distillation and mechanism transfer; graph distillation as a supernode-level structural loss on attribution graphs; the Llama-3.1-8B  $\rightarrow$  Llama-3.2-1B arithmetic setting; headline result.]*

## 1 INTRODUCTION

*[Introduction. Why matching outputs need not transfer the teacher’s algorithm; why attribution graphs give a model-agnostic handle on the mechanism; contributions.]*

## 2 RELATED WORK

*[Related work. Knowledge distillation and feature/representation matching; mechanistic interpretability and circuit discovery (attribution patching, transcoders, attribution graphs); prior attempts at mechanism-level transfer.]*

## 3 METHOD

### 3.1 OVERVIEW

Standard knowledge distillation constrains only the *output* of the student and says nothing about how that output was produced (Hinton et al., 2015). We add a second, orthogonal constraint on the *internal computation* (Appendix A discusses in what sense it is orthogonal to the output-level loss): for a given arithmetic prompt we build an attribution graph over MLP neurons in each model, coarsen it into a small set of functionally-labelled *supernodes* that exist in both models, and require the teacher and the student to route causal influence between those supernodes in the same proportions.

Let  $\mathcal{T}$  be the frozen teacher (Llama-3.1-8B) and  $\mathcal{S}$  the student (Llama-3.2-1B) with trainable parameters  $\theta_S$ . For a prompt  $x$ , the pipeline of Sections 3.2–3.4 produces one  $K \times K$  supernode adjacency matrix per model,  $W^{\mathcal{T}}(x)$  and  $W^{\mathcal{S}}(x; \theta_S)$ , whose row  $t$  says *where supernode  $t$  draws its input from*. Supernodes are identified by a semantic label (“ones digit of the second operand”, “magnitude of the sum”, ...) rather than by neuron index, so the rows and columns of the two matrices correspond even though the models differ in width, depth, and neuron basis.

Dividing a row by its  $\ell_1$  norm turns it into a distribution over sources — supernode  $t$ ’s *routing profile*, the fraction of its incoming causal influence contributed by each other supernode. Writing  $P_t$  and  $Q_t$  for the teacher’s and the student’s routing profile for supernode  $t$ , and  $\mathcal{A}$  for the supernodes present in both models, the graph loss is the mean Jensen–Shannon divergence between matched profiles,

$$\mathcal{L}_{\text{graph}}(\theta_S) = \frac{1}{|\mathcal{A}|} \sum_{t \in \mathcal{A}} \text{JSD}(P_t \parallel Q_t), \quad \text{JSD}(P \parallel Q) = \frac{1}{2} \text{KL}(P \parallel M) + \frac{1}{2} \text{KL}(Q \parallel M), \quad (1)$$

with  $M = \frac{1}{2}(P + Q)$  and  $P_t$  detached, since the teacher is fixed. Unlike a one-sided KL, the JSD is symmetric — the student is penalised both for missing an edge the teacher uses and for inventing one the teacher does not — and bounded, so a single edge on which the two models place disjoint mass cannot dominate the loss.

The full objective adds (1) to the ordinary temperature-scaled KD term,

$$\mathcal{L}(\theta_S) = \underbrace{\frac{\tau^2}{|\mathcal{P}|} \sum_{p \in \mathcal{P}} \text{KL}\left(\text{softmax}(z_p^T/\tau) \parallel \text{softmax}(z_p^S/\tau)\right)}_{\mathcal{L}_{\text{KD}}} + \lambda \mathcal{L}_{\text{graph}}(\theta_S), \quad (2)$$

where  $z_p$  are logits at non-padding position  $p$ ,  $\tau = 2$ , and  $\lambda = 1$ .

### 3.2 ATTRIBUTION GRAPHS OVER MLP NEURONS

Our graph construction follows the circuit-tracing methodology of Ameisen et al. (2025); Lindsey et al. (2025): represent one forward pass as a directed weighted graph whose nodes are localised computational units — active at a specific layer and token position — and whose edge  $A_{ij}$  measures the *direct* linear effect of node  $j$  on node  $i$ , holding fixed everything off the direct path.

A node  $i$  is a triple  $(\ell_i, p_i, n_i)$ : layer, token position, neuron index. It carries a *write vector*  $v_i^{\text{out}} = a_i \cdot W_{\text{down}}[:, n_i]$ , what the neuron adds to the residual stream, and a *read direction*  $v_i^{\text{in}} = \partial a_i / \partial h$ , the residual-stream direction it responds to. Edges are gradient-activation products, i.e. attribution patching (Nanda, 2023): for a target node  $i$  we form the scalar  $f_i = \langle h_{p_i}^{(\ell_i)}, \text{sg}[v_i^{\text{in}}] \rangle$  — the linearised pre-activation of neuron  $i$ , with the read direction stop-graded so that only the *input* to the neuron is differentiated — and set

$$A_{ij} = \left\langle \nabla_{m_{p_j}^{(\ell_j)}} f_i, v_j^{\text{out}} \right\rangle, \quad (3)$$

where  $m^{(\ell_j)}$  is the output of MLP block  $\ell_j$ . Token-embedding nodes are handled the same way with  $v^{\text{out}}$  replaced by the embedding vector, and logit nodes use  $f_c = \langle h_{-1}^{(L)}, W_U[:, c] - \overline{W_U} \rangle$  with the mean-subtracted unembedding column, following Ameisen et al. (2025). The logit targets  $c$  are the smallest set of top tokens whose probabilities under  $\text{softmax}(z/\tau)$ ,  $\tau = 2$ , sum to 0.95, capped at ten. Since  $n_{\text{pos}} \times d_{\text{mlp}}$  candidate neurons per layer is far too many, we keep the top 10% per layer by write-vector norm; activations at the BOS position are zeroed so the attention sink is never selected.

*Why raw neurons are enough.* The published pipeline attributes to cross-layer transcoder features rather than to neurons (Dunefsky et al., 2024). We attribute directly to raw MLP neurons: the construction (3) only requires a node’s residual-stream contribution to be linear in its activation, which neurons satisfy exactly, and it avoids the transcoder’s approximation error, which must otherwise be carried as uninterpretable *error nodes*. The remaining cost is polysemanticity, and that is precisely what the supernode step repairs — in the published work, too, the interpretable unit is a hand-built group of features rather than the individual feature. We automate that grouping (Section 3.3), and the loss (1) is defined over groups, never over individual nodes.

### 3.3 GROUPING NEURONS INTO SUPERNODES

Raw neuron graphs are far too large and too model-specific to compare across a 1B and an 8B model. Where supernodes in the published work are drawn by hand from feature visualisations, we replace the manual step with an automatic, purely functional criterion, which is what makes the grouping reproducible across two different models. We first describe the criterion concretely on our task (Section 3.3.1), then the task-agnostic construction it instantiates (Section 3.3.2).

#### 3.3.1 CASE STUDY: ANOVA SUPERNODES FOR TWO-DIGIT ADDITION

Every neuron is evaluated on a fixed probe set: all two-digit addition prompts “ $a+b=$ ” for  $a, b \in \{0, \dots, 99\}$ , i.e. all  $10^4$  points of the  $100 \times 100$  grid. This yields for neuron  $i$  an *activation grid*  $T_i \in \mathbb{R}^{100 \times 100}$ , with  $T_i[a, b]$  its activation on that prompt — a direct empirical description of what the neuron responds to, as a picture. The node panels of Figure 1 are examples.

We then ask which of a fixed set of arithmetically meaningful partitions of the grid best explains that picture. Each candidate is a binary mask  $M \in \{0, 1\}^{100 \times 100}$ ; for a prompt with operands  $(a^*, b^*)$  and sum  $s^* = a^* + b^*$ , the categories are

| category   | mask $M[a, b] = 1$ iff   | category                  | mask $M[a, b] = 1$ iff       |
|------------|--------------------------|---------------------------|------------------------------|
| arg1 range | $a = a^*$                | sum range                 | $a + b = s^*$                |
| arg1 units | $a \equiv a^* \pmod{10}$ | sum units                 | $a + b \equiv s^* \pmod{10}$ |
| arg2 range | $b = b^*$                | arg1 units and arg2 units | composite (see below)        |
| arg2 units | $b \equiv b^* \pmod{10}$ |                           |                              |

The score of mask  $M$  for neuron  $i$  is the fraction of the variance of  $T_i$  that the mask explains. For a binary regressor this is the squared Pearson correlation, and is exactly the one-way ANOVA  $\eta^2$  between the two groups the mask induces:

$$s_{i,M} = \frac{(\langle T_i - \bar{T}_i, M - \bar{M} \rangle)^2}{\|T_i - \bar{T}_i\|_2^2 \cdot \|M - \bar{M}\|_2^2} \in [0, 1], \quad (4)$$

and a category’s score  $s_i^c$  is the best score over its masks. The composite category `arg1 units` and `arg2 units` instead scores  $\min(s_i^{\text{arg1 units}}, s_i^{\text{arg2 units}})$ , so it fires only for a neuron sensitive to both ones digits at once. Neurons are ranked not by  $s_i^c$  but by *specificity*, the margin over the best competing category,  $\delta_i^c = s_i^c - \max_{c' \neq c} s_i^{c'}$ , and supernode  $S_c$  is the top  $n_{\text{per}} = 10$  neurons by  $\delta_i^c$ . Ranking by margin is what keeps the supernodes functionally disjoint: a neuron firing on a broad band of  $a$  scores well on several correlated categories at once, and only the margin separates a genuine ones-digit detector from a magnitude detector that happens to correlate with one.

Two categories are handled differently, because their neurons are defined by what they *write* rather than by what they read. For `sum range` and `sum units` we rank candidates by how closely the neuron’s direct logit attribution  $\text{softmax}(v_i^{\text{out}\top} W_U)$ , restricted to the number tokens 0–200, matches the target distribution in KL — a point mass at  $s^*$  for `sum range`, and uniform over  $\{n : n \equiv s^* \pmod{10}\}$  for `sum units`. The same score defines two further `dla` supernodes, one per sum target, collecting the neurons that write those distributions to the output most directly; these are the bottom two nodes on each side of Figure 1.

Because the same category list produces the same supernode labels in both models, the teacher and student supergraphs are aligned automatically; training requires the two label sets to match exactly and falls back to their intersection with a warning if they do not.

### 3.3.2 THE GENERAL RECIPE: SUPERNODES FROM LOGIT AND ARGUMENT ATTRIBUTION

The ANOVA categories above are hand-written for two-digit addition, but nothing downstream of the grouping depends on where the labels come from: the loss (1) only needs supernode labels that are computable in both models and identical across them. Our general construction derives the labels from the prompt itself — one supernode per input token, plus one for the output distribution — so it transfers to a new task with no masks or probe grids to design.

*Argument supernodes.* For every token position  $q$  (BOS excluded) with embedding  $E_q$ , neuron  $i$  is scored by  $|\langle v_i^{\text{in}}, E_q \rangle|$  — an inner-product proxy for the direct attribution edge (3) from the token-embedding node at  $q$  into the neuron’s linearised pre-activation, costing no extra backward pass. Each neuron’s scores are then normalised across positions into a distribution, so a neuron counts toward a position only through the *share* of its total embedding alignment concentrated there; this favours neurons selectively tied to one token over neurons whose read direction is simply large. The supernode for position  $q$  is the top  $n_{\text{per}}$  neurons by that share, and its label is the token string itself (`arg:36, arg:+, arg:59, arg:=`).

*The output supernode.* At the answer position, each neuron’s direct logit attribution  $\text{softmax}(v_i^{\text{out}\top} W_U)$  is scored by its KL divergence from the reference output distribution  $\text{softmax}(z/\tau)$ , restricted to the reference’s top 100 tokens and renormalised. The  $n_{\text{per}}$  closest neurons form a single `dla` supernode — the neurons that most directly write the answer distribution — generalising the DLA ranking used for the sum categories above from a hand-specified target to the model’s actual output. During training the reference logits  $z$  are the teacher’s for both models (Appendix B), which anchors the student’s `dla` supernode to a stationary target.

Every label is derived from the prompt tokens and reference logits the two models share, so the teacher and student supergraphs align automatically on any prompt. This construction is the default

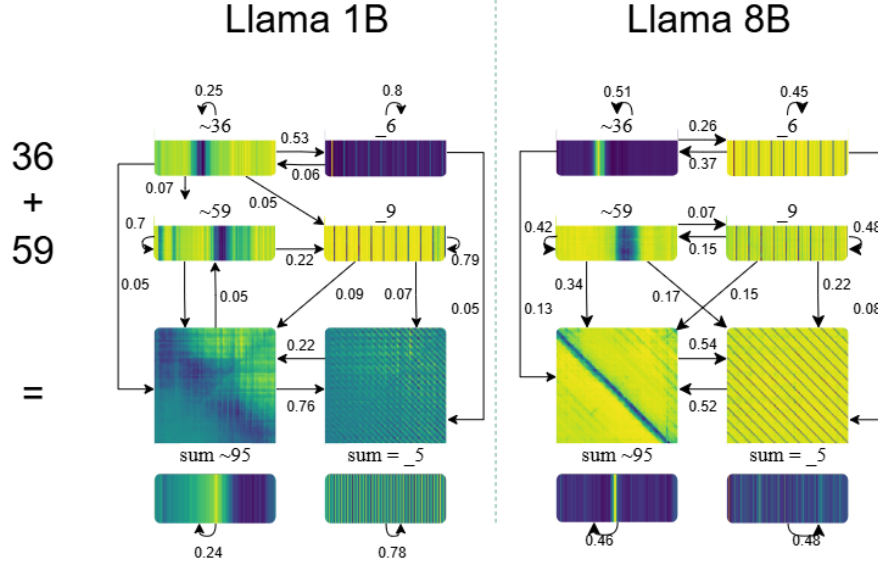


Figure 1: Supergraphs for the prompt “36+59=” in Llama-3.2-1B (left) and Llama-3.1-8B (right). Each node is a supernode from Section 3.3, drawn as the activation grid  $T_i$  of one representative member neuron — its activation across all  $10^4$  two-digit addition prompts, with operand  $a$  on one axis and  $b$  on the other — so that membership and computational role can be read off the same picture. Nodes  $\sim 36$  and  $\sim 59$  are `arg1/arg2_range`, `_6` and `_9` are `arg1/arg2` units, `sum  $\sim 95$`  and `sum =  $_5$`  are `sum_range` and `sum` units, and the bottom pair are the corresponding `dla` supernodes. Edge labels are the supernode weights of (5); self-loops are within-supernode recurrence. The 8B grids are visibly cleaner: its `sum  $\sim 95$`  node is a sharp anti-diagonal ( $a + b \approx 95$ ) and its `sum =  $_5$`  node a clean periodic lattice ( $a + b \equiv 5 \pmod{10}$ ), whereas the 1B counterparts are diffuse and noisy — the same structure, represented less separably in the smaller model.

in our training runs; the ANOVA categories of Section 3.3.1 remain available as an opt-in whitelist, and produce the task-level supergraphs of Figure 1.

### 3.4 SUPERNODE EDGE WEIGHTS

The supergraph’s edges are the supernode adjacency of Ameisen et al. (2025),

$$\text{supernode\_adjacency}(t, s) = \frac{\sum_{n_t \in N_t} \text{frac\_external}(n_t) \cdot \sum_{n_s \in N_s} A'_{n_t, n_s}}{\sum_{n_t \in N_t} \text{frac\_external}(n_t)}, \quad (5)$$

where  $N_t$  and  $N_s$  are the member nodes of supernodes  $t$  and  $s$ , and  $A'$  is the graph adjacency with each node’s incoming weights normalised to sum to one over *all* sources — neurons, token embeddings and logits alike. The weight  $\text{frac\_external}(n_t)$  is the fraction of a member’s absolute inbound weight that comes from outside its own supernode, so members that mainly feed one another count for little and a supernode cannot dominate its own inbound profile. Because  $A'$  normalises over all nodes while the supergraph spans only supernode columns, the rows of `supernode_adjacency` do not sum to one, which is why the routing profiles of Section 3.1 renormalise them.

Implementation details — adapting the attribution pipeline to Llama’s gated MLPs and Hugging Face autograd, and the optimisations that make it fast enough for a training loop — are given in Appendices B and E.

## 4 EXPERIMENTAL SETUP

*[Experimental setup. Teacher/student pair and checkpoints; training data and splits; optimiser, schedule, batch size,  $n_{\text{graph}}$ ; which supernode categories are enabled; baselines (SFT, standard KD, KD + graph loss); evaluation metric and prompt format; hardware and wall-clock cost per step.]*

## 5 RESULTS

*[Results. Accuracy against the baselines; graph loss and KD loss curves; whether supernode routing actually converges toward the teacher's; per-category breakdown of which supernodes align and which do not.]*

## 6 ANALYSIS

*[Analysis. Qualitative before/after supergraphs for the student; do the student's activation grids sharpen over training? Ablations:  $\lambda$ , JSD vs. other divergences, number of supernodes,  $n_{\text{per}}$ , and the linearisation ablation of Appendix C.]*

## 7 LIMITATIONS

*[Limitations. Single task family (two-digit addition) and a single teacher/student pair; attribution graphs are built per prompt and only at sampled positions; edge sign is discarded by the input normalisation; supernode membership is recomputed but not itself differentiable.]*

## 8 CONCLUSION

*[Conclusion.]*

### AI USE STATEMENT

*[AI use statement (required by ICLR 2027; see the AI Policy for Authors). State which of the tasks with required disclosure generative AI was used for and which it was not, how AI-assisted work was reviewed, and that the authors take responsibility for the final content.]*

### ETHICS STATEMENT

*[Ethics statement (recommended). Synthetic arithmetic data, public model checkpoints, no human subjects; note anything the Code of Ethics asks about.]*

### REPRODUCIBILITY STATEMENT

*[Reproducibility statement (recommended). Point to the appendices that carry the full hyperparameters, the training precision regime, the supernode construction and the diagnostics, and to the anonymised code in the supplementary material.]*

## REFERENCES

- Emmanuel Ameisen, Jack Lindsey, Adam Pearce, Wes Gurnee, Nicholas L. Turner, Brian Chen, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Thomas Conerly, Joshua Batson, Chris Olah, and Tom Henighan. Circuit tracing: Revealing computational graphs in language models. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/methods.html>.
- Jacob Dunefsky, Philippe Chlenski, and Neel Nanda. Transcoders find interpretable LLM feature circuits. *Advances in Neural Information Processing Systems*, 2024.
- Geoffrey Hinton, Oriol Vinyals, and Jeff Dean. Distilling the knowledge in a neural network, 2015.

Jack Lindsey, Wes Gurnee, Emmanuel Ameisen, Brian Chen, Adam Pearce, Nicholas L. Turner, Craig Citro, David Abrahams, Shan Carter, Basil Hosmer, Jonathan Marcus, Michael Sklar, Adly Templeton, Trenton Bricken, Callum McDougall, Thomas Conerly, Joshua Batson, Chris Olah, and Tom Henighan. On the biology of a large language model. *Transformer Circuits Thread*, 2025. URL <https://transformer-circuits.pub/2025/attribution-graphs/biology.html>.

Neel Nanda. Attribution patching: Activation patching at industrial scale. <https://www.neelnanda.io/mechanistic-interpretability/attribution-patching>, 2023.

Noam Shazeer. GLU variants improve transformer. In *arXiv preprint arXiv:2002.05202*, 2020.

## A ORTHOGONALITY OF THE GRAPH AND KD LOSSES

The KD loss constrains only the output distribution and is invariant to which internal circuit produces it; the graph loss constrains routing between supernodes and is close to invariant to the output. If that separation is real, the two gradients should be close to orthogonal in parameter space rather than one being a reparameterisation of the other.

We measure this directly (Figure 2). At the student’s initial checkpoint we compute  $\nabla_{\theta_S} \mathcal{L}_{\text{KD}}$  over one batch of 32 prompts and  $\nabla_{\theta_S} \mathcal{L}_{\text{graph}}$  over 8 prompts, and take the cosine between them. Across all four linearisation variants of Appendix C the cosine lies in  $[-0.024, -0.008]$ : every variant lies between  $90.5^\circ$  and  $91.4^\circ$ , and the near-orthogonality is a property of the objective rather than of one particular linearisation.

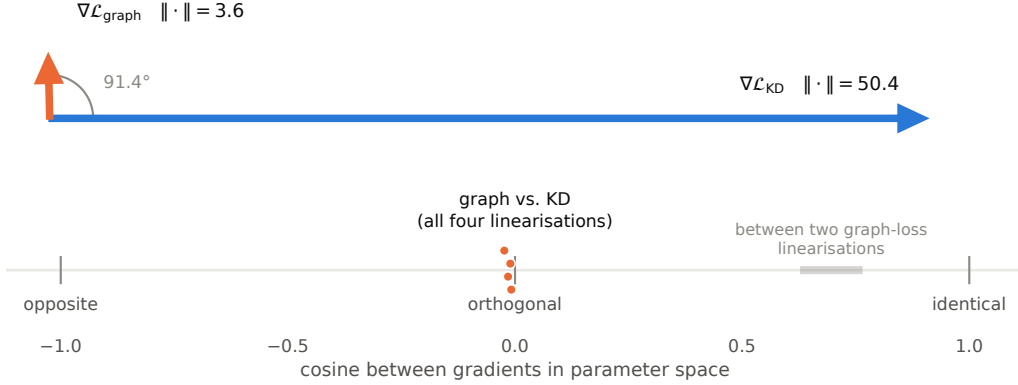


Figure 2: The graph gradient is both small and sideways. *Above*: the two gradients drawn to scale on equal axes, so the angle is not distorted; the arc is the unfrozen variant, and the other three linearisations lie between  $90.5^\circ$  and  $91.4^\circ$ . *Below*: the same cosines on a  $[-1, 1]$  line. The four points are offset vertically only to separate values that coincide to within 0.015; the offset carries no meaning. The grey bar is the same measure taken between two graph-loss linearisations (0.63 to 0.77, Appendix C), included because without a yardstick a reader cannot tell whether  $-0.02$  is near zero or merely small.

This also resolves an apparent puzzle about magnitudes. At  $\lambda_{\text{graph}} = 1$  the graph gradient has norm 3.65 against the KD gradient’s 50.41, so at the  $\lambda_{\text{graph}}$  actually used in training the graph term is smaller by more than two orders of magnitude, yet it changes downstream accuracy substantially (Appendix C). Orthogonality explains why: AdamW normalises per parameter, so in the large subspace where the KD gradient is near zero the graph term is the only signal present and receives a full-size step regardless of its global norm. The graph loss does not accelerate KD along KD’s own direction — there is no such component to accelerate — it moves the student in directions output-level distillation never explores.

es: Measured at initialisation only. Tracking the cosine over training, and on a second teacher/student pair, is still to do.

## B ADAPTING THE PIPELINE TO LLAMA

The published methodology assumes a transcoder replacement model and a TransformerLens-style hook API (Ameisen et al., 2025). Running it inside a Hugging Face training loop on Llama required several changes, most forced by the requirement that the student’s graph carry gradients back into  $\theta_S$ .

*Gated MLPs.* Llama’s MLP is a SwiGLU block (Shazeer, 2020) with no single input matrix. With residual-stream input  $h$  at the node’s position,

$$a_i = \sigma(g_i) u_i, \quad v_i^{\text{in}} = \frac{\partial a_i}{\partial h} = \sigma(g_i) W_{\text{up}}[n_i, :] + u_i \sigma'(g_i) W_{\text{gate}}[n_i, :], \quad (6)$$

where  $g_i = \langle W_{\text{gate}}[n_i, :], h \rangle$ ,  $u_i = \langle W_{\text{up}}[n_i, :], h \rangle$  and  $\sigma$  is SiLU. The read direction is therefore a product-rule sum of two terms that itself depends on the activation, rather than a fixed row of  $W_{\text{in}}$ .

*Differentiability.* The pre-selection top- $k$  is detached (indices are not differentiable), but activations and read/write vectors stay in the ambient autograd context, so gradients flow  $\mathcal{L}_{\text{graph}} \rightarrow \text{supernode\_adjacency} \rightarrow A' \rightarrow A \rightarrow v^{\text{out}} \rightarrow W_{\text{down}}$  back into the model. Supernode *membership* is recomputed each step but treated as fixed structure; only the aggregation (5) is rebuilt differentially, out-of-place, so that autograd views survive.

*Shared attribution targets.* Both models are attributed to the *same* logit token set — the teacher’s — and the student’s `dla` supernodes are selected against the teacher’s logits. Otherwise the two graphs would be built over different output nodes, and would be non-stationary early in training when the student’s own output distribution is still wrong.

*Linearisation.* The published method freezes attention patterns and normalisation denominators when linearising, so that edges reflect only the direct residual-stream path. We implement both freezes as backward-only stop-gradients and make each independently selectable, so (3) can be evaluated either with the direct-path linearisation or through the full Hugging Face Jacobian. Appendix C measures what the choice costs: it changes the gradient’s direction rather than its magnitude, and the direct-path linearisation is the better training signal on every metric we tracked.

*Precision.* The graph gradient is small next to the KD gradient — 0.5–6% of its norm for  $\lambda \in [0.01, 0.3]$  (Appendix D) — and it arrives one prompt at a time, since each prompt’s graph is backpropagated and freed before the next is built (Appendix E). With bf16 parameters those  $\sim 32$  partial sums each land below half a unit in the last place of a `.grad` buffer that already holds the KD gradient, and are rounded away: we measured only 66–100% of the graph gradient surviving accumulation, decreasing with  $\lambda$ , which silently compressed a  $\lambda$  sweep toward its top. bf16 weights lose the update itself as well, since an AdamW step at  $10^{-6}$ – $10^{-5}$  is under half a ULP for most weights. The student therefore holds fp32 master weights and fp32 optimiser moments and runs its forward under bf16 autocast; the inference-only teacher stays bf16. The test that this is enough is the step-one check of Appendix D: summaries of the graph gradient that are invariant to its scale must agree across  $\lambda$ , and now do.

## C THE DIRECT-PATH LINEARISATION AS A TRAINING SIGNAL

Circuit tracing freezes attention patterns and RMSNorm denominators so that an edge is the contribution of the direct path alone (Ameisen et al., 2025). That choice is made for interpretability: it renders the graph locally linear in the node activations and therefore readable. It is not obvious that it is also the right choice when the graph is a regression target rather than a figure, since the frozen edge omits real causal influence — paths on which a source node shifts *where* a target attends, and the rank-one RMSNorm term that projects out the component of each edge along the activation direction. This appendix measures the difference.

### C.1 IMPLEMENTATION

Both freezes are backward-only. Detaching a multiplicative factor leaves every forward value bit-identical, so activations, read and write vectors, neuron pre-selection and supernode membership are unchanged; only the gradients that become edge weights differ. Freezing attention additionally requires a kernel that materialises the pattern, since the default fused kernel never forms it.

One implementation detail is load-bearing. Attribution involves two forward passes: one whose captured MLP inputs carry gradient back to  $\theta_S$ , and one whose *backward* produces the edge values of (3). The freeze belongs only on the second. Applied to the first it cannot change any edge — being backward-only, it changes no forward value — but it silently biases training, zeroing the gradient to  $W_Q$  and  $W_K$  outright and stripping the rank-one correction from every RMSNorm on the path back to the weights. We hit this: with the freeze on both passes, in-distribution accuracy collapsed from 0.634 to 0.090 within twenty steps while the KD loss continued to fall.

### C.2 GRADIENT GEOMETRY

Table 1 reports the graph gradient under each variant at the student’s initial checkpoint, against the unfrozen gradient and against the KD gradient.

Table 1: Graph-loss gradient under each linearisation, at the student’s initial checkpoint; 22\_add, 8 graph prompts,  $\lambda_{\text{graph}} = 1$ . Cosines are against the unfrozen graph gradient and against the KD gradient.

|                           | $\mathcal{L}$ | $\ g\ $ | $\ g\ /\ g_{\text{unfr}}\ $ | $\cos(g, g_{\text{unfr}})$ | $\cos(g, g_{\text{KD}})$ |
|---------------------------|---------------|---------|-----------------------------|----------------------------|--------------------------|
| $\mathcal{L}_{\text{KD}}$ | 2.0254        | 50.41   | —                           | —                          | —                        |
| unfrozen                  | 0.3405        | 3.647   | 1.000                       | +1.0000                    | −0.0236                  |
| attention frozen          | 0.3501        | 4.059   | 1.113                       | +0.6276                    | −0.0103                  |
| RMSNorm frozen            | 0.3722        | 3.991   | 1.094                       | +0.7652                    | −0.0153                  |
| both frozen               | 0.3745        | 3.856   | 1.057                       | +0.6984                    | −0.0082                  |

Two things stand out. First, the freeze changes direction, not strength: every frozen variant is  $40^\circ$  to  $51^\circ$  away from the unfrozen gradient while being *larger* in norm. The unfrozen total-effect gradient is the smallest of the four, which is what destructive interference between paths produces — freezing removes the cancellation rather than removing signal. Second, the two freezes partially cancel each other: freezing both gives a smaller gradient than either alone and an angle between the two, so their deviations from the unfrozen gradient are anti-correlated. A plausible mechanism is that the RMSNorm freeze injects radial (pure-scale) sensitivity that the true Jacobian projects out, while the attention freeze deletes the QK path, which is the channel most responsive to scale because a radial perturbation rescales the query and so sharpens or flattens the softmax.

### C.3 TRAINING ABLATION

Table 2 trains all four variants for 15 steps under otherwise identical settings, evaluating in-distribution on two-digit addition and out-of-distribution on four arithmetic families never seen in training.

Figure 3 plots the trajectories. Both metrics rank the four variants identically and monotonically: freezing more is better, and the direct-path linearisation is best. The RMSNorm freeze is the large effect (+0.091 OOD on its own, +0.100 on top of the attention freeze) and the attention freeze a small consistent bonus (+0.002 alone, +0.011 on top of RMSNorm); the two are mildly super-additive on OOD, since additivity would predict 0.3496 against the 0.3582 observed.

The sharpest evidence is a sign change rather than a difference of means. On 33\_add and 2222\_add, the two variants without the RMSNorm freeze end *below* the untrained student (0.2686 and 0.2621 against a 0.3189 baseline; 0.0206 and 0.0210 against 0.0236), while both variants with it end above. Without the RMSNorm freeze, graph distillation degrades those two families; with it, they improve.

Table 2: Distillation accuracy under each linearisation. Peak column is the maximum in-distribution accuracy over the run; remaining columns are at step 15. OOD is the mean of the four right-hand columns. Single seed.

|                    | 22_add (peak) | OOD           | 222_add       | 2222_add      | 33_add        | 21_mult       |
|--------------------|---------------|---------------|---------------|---------------|---------------|---------------|
| student, untrained | 0.6340        | 0.1395        | 0.1034        | 0.0236        | 0.3189        | 0.1120        |
| teacher            | 0.9562        | —             | —             | —             | —             | —             |
| unfrozen           | 0.9588        | 0.2560        | 0.1698        | 0.0206        | 0.2686        | 0.5650        |
| attention frozen   | 0.9678        | 0.2582        | 0.1748        | 0.0210        | 0.2621        | 0.5750        |
| RMSNorm frozen     | 0.9792        | 0.3474        | 0.2681        | 0.0298        | 0.3598        | 0.7320        |
| both frozen        | <b>0.9824</b> | <b>0.3582</b> | <b>0.2840</b> | <b>0.0352</b> | <b>0.3714</b> | <b>0.7420</b> |

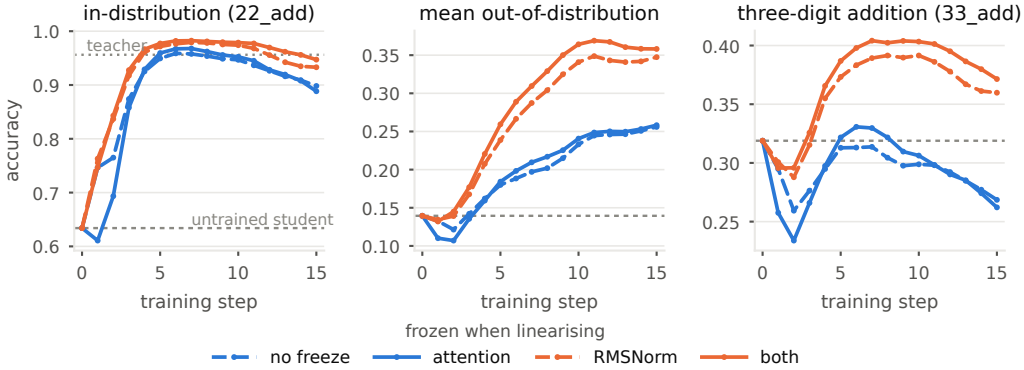


Figure 3: Accuracy against training step for the four linearisations. Hue carries the RMSNorm freeze and line style the attention freeze, so the  $2 \times 2$  design is visible as two pairs: the RMSNorm-frozen runs separate from the other two within a few steps and stay separated. The dashed rule marks the untrained student on every panel and the dotted rule in the left panel marks the teacher. In-distribution accuracy peaks around step 6–7 and then declines in all four runs, while out-of-distribution accuracy is still climbing at step 15 for the two runs without the RMSNorm freeze; the right panel shows those same two runs ending *below* the untrained student on three-digit addition, which the two RMSNorm-frozen runs do not.

#### C.4 WHAT PREDICTS QUALITY, AND WHAT DOES NOT

Cosine to the unfrozen gradient does *not* predict downstream quality, and is close to anti-predictive: it ranks the unfrozen gradient highest by construction, and the unfrozen variant is the worst of the four. Proximity to the total-effect gradient is not the property that matters.

The initial value of  $\mathcal{L}_{\text{graph}}$  does order all four variants correctly on both metrics — the harder the teacher’s supergraph is to match at initialisation, the better the student ends up — which is consistent with the interference account above, since the freezes expose structure that path cancellation averages away in the total effect. With four points this is an observation rather than a result, but it is free: the quantity is already in the training log at step one.

#### C.5 GRADIENT DYNAMICS OVER TRAINING

All four runs were repeated with per-step gradient instrumentation. Training is seeded and the data order is deterministic, so accuracy reproduced bit-for-bit and the instrumentation is free. Figure 4 plots the result and Table 3 summarises it.

*Orthogonality is maintained, not merely initial.* The step-one cosine is  $-0.0237$ , against the  $-0.0236$  that Appendix A measures by a separate route at the same checkpoint. Over fifteen steps it oscillates within roughly  $[-0.033, +0.049]$  in every variant and never drifts, so the mean absolute cosine stays

below 0.006. The near-orthogonality of  $\mathcal{L}_{\text{graph}}$  and  $\mathcal{L}_{\text{KD}}$  is therefore a property of the objective throughout training rather than an artefact of initialisation.

*The graph term progressively takes over.*  $\|\nabla \mathcal{L}_{\text{KD}}\|$  falls by a factor of 14.6 over fifteen steps, from 50.4 to 3.5, while  $\|\nabla \mathcal{L}_{\text{graph}}\|$  stays flat near 1.5. Their ratio therefore climbs from about 0.05 to between 0.38 and 0.53, and the fraction of parameter entries whose update sign the graph term decides rises from 0.02 to 0.12. Nobody chose this schedule: it is a consequence of KD converging faster than the graph objective.

It also lines up with the accuracy curves. In-distribution accuracy peaks at step 6–7, where the ratio is around 0.10–0.15, and declines from there, while out-of-distribution accuracy goes on rising to step 11–15 (Figure 3). As the graph term gains relative weight, the student trades in-distribution fit for out-of-distribution generalisation — which is what two near-orthogonal objectives should do, since a single trajectory cannot maximise both.

Table 3: Per-step gradient metrics, averaged over the fifteen steps. The graph norm is already scaled by  $\lambda_{\text{graph}}$ , so these are the quantities that reach the optimiser.

|                        | $\ g_{\text{KD}}\ $ | $\ g_{\text{graph}}\ $ | ratio | cos     | sign flips |
|------------------------|---------------------|------------------------|-------|---------|------------|
| unfrozen               | 15.62               | 1.726                  | 0.184 | +0.0053 | 0.0750     |
| attention frozen       | 15.62               | 1.579                  | 0.175 | +0.0042 | 0.0712     |
| RMSNorm frozen         | 15.66               | 1.762                  | 0.189 | +0.0025 | 0.0747     |
| both frozen            | 15.63               | 1.887                  | 0.213 | +0.0007 | 0.0774     |
| spread across variants | 0.3%                | 17.7%                  | 20.1% | —       | 8.3%       |

*No aggregate gradient statistic separates the four variants.* This is the substantive negative result of the instrumentation. The linearisations differ by 33.5% in relative out-of-distribution accuracy, but by 0.3% in KD gradient norm — as they must, being the same objective — and by only 8.3% in sign flips and 20.1% in ratio. Two of the metrics carry orderings that are simply wrong: sign flips place the unfrozen run, the worst performer, second. The cosine with the KD gradient — a different quantity from the cosine with the unfrozen gradient in Section C.4 — is the one whose ordering matches accuracy for all four variants, more orthogonal being better, but the spread between runs (0.0047) is smaller than the standard error implied by the step-to-step scatter within a single run (0.0055), so it cannot be claimed.

Whatever distinguishes the linearisations is therefore not the size of the graph gradient, nor its alignment with the KD gradient, but which parameters it moves — and scalar summaries are blind to that by construction. The per-group figures in Table 1 point the same way: the RMSNorm freeze almost doubles the gradient reaching the normalisation weights (ratio 1.93) while the global norm barely moves at all.

One incidental observation: the pre-clip gradient norm falls from 50.5 to 3.8 and so exceeds the clip threshold of 1.0 at every step of every run. Gradient clipping is not a rarely-triggered safety valve here but a permanent rescaling — one that AdamW largely undoes, since per-parameter normalisation cancels a uniform scale factor.

## C.6 CAVEATS

Single seed, 15 steps, and a single supernode construction — the arg-token and DLA supernodes of Section 3.3.2, not the ANOVA supernodes. The freeze acts on the edge coefficients, before supernodes are formed, so the RMSNorm result should transfer; the attention result is less safe to extrapolate, since arg-token supernodes are positional and attention is the only mechanism that moves information between positions. In-distribution accuracy peaks at step 6–7 in all four runs and declines afterwards, while OOD accuracy is still rising at step 15 for the two variants without the RMSNorm freeze, so their OOD numbers are lower bounds. The gap between the two best variants (0.011 OOD) is within what a single seed can plausibly produce; the gap between the RMSNorm-frozen and unfrozen variants (0.091 OOD, plus the two sign changes) is not. 2222\_add is close to unlearned in every configuration. es: Seeds and longer runs before this table goes in the camera-ready.

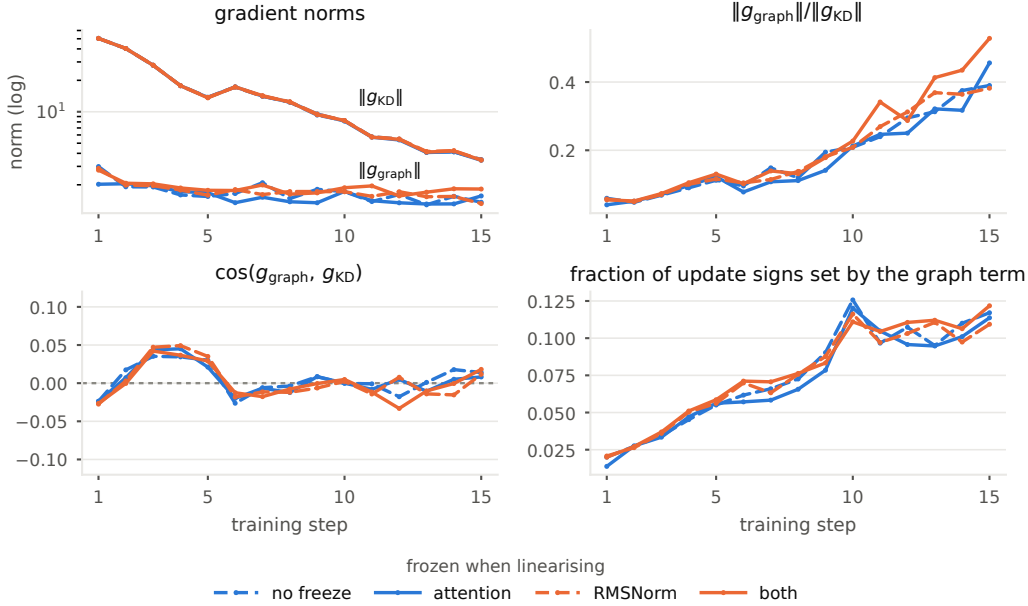


Figure 4: Per-step gradient metrics for the four linearisations, encoded as in Figure 3. *All four variants are drawn on every panel and lie almost exactly on top of one another* — that overlap is the point, not a plotting artefact. The KD gradient collapses by a factor of 14.6 while the graph gradient stays flat, so their ratio climbs tenfold and the share of update signs the graph term decides rises from 2% to 12%, none of it by design. The cosine oscillates about zero throughout and never drifts. Yet these four runs differ by a third in out-of-distribution accuracy, so what separates them is not visible in any of these aggregates.

## D SENSITIVITY TO THE GRAPH-LOSS WEIGHT

The weight  $\lambda$  in (2) is the one hyperparameter the method adds. We sweep  $\lambda \in \{0.01, 0.1, 0.3\}$  against a  $\lambda = 0$  control, which is standard knowledge distillation through the same pipeline, with everything else held fixed: learning rate  $10^{-6}$ , batch size 32,  $\tau = 2$ , a shared seed so every run sees the same batches, and the unfrozen arg-token and DLA supernodes of Section 3.3.2 with three neurons each. The graph runs train for 100 steps; the control trains for 300 so that its peak is not cut off. Table 4 and Figure 5 report the result.

Table 4: Accuracy under each graph-loss weight, single seed. Peak columns are the maximum over the run, with the step in parentheses; family columns are at step 100; *dip* is the minimum in-distribution accuracy over steps 1–30 and *recovery* the first step back at or above the untrained student. The control ran to step 300. Entries below the untrained student are in italics.

|                           | 22_add (peak)      | OOD (peak)          | OOD (100)     | 222_add       | 2222_add      | 33_add        | 21_mult       | dip / recovery    |
|---------------------------|--------------------|---------------------|---------------|---------------|---------------|---------------|---------------|-------------------|
| student, untrained        | 0.6404             | 0.1421              | —             | 0.1050        | 0.0244        | 0.3199        | 0.1190        | —                 |
| KD only ( $\lambda = 0$ ) | 0.9376 (195)       | 0.1928 (195)        | 0.1587        | <i>0.0973</i> | <i>0.0072</i> | <i>0.1923</i> | 0.3380        | 0.180 / 70        |
| $\lambda = 0.01$          | 0.9732 (100)       | 0.2569 (98)         | 0.2564        | 0.2083        | <i>0.0174</i> | <i>0.2930</i> | 0.5070        | 0.207 / 55        |
| $\lambda = 0.1$           | <b>0.9862</b> (98) | <b>0.3774</b> (100) | <b>0.3774</b> | <b>0.3845</b> | 0.0614        | <b>0.3807</b> | <b>0.6830</b> | 0.384 / 28        |
| $\lambda = 0.3$           | 0.9570 (76)        | 0.3684 (84)         | 0.3362        | 0.3032        | <b>0.0696</b> | 0.3391        | 0.6330        | <b>0.547 / 18</b> |

Three things hold across every metric and every family. First, output-level distillation alone does not generalise in this setting: the control peaks at 0.938 in-distribution and 0.193 mean OOD, and at no point in 300 steps recovers the untrained student on three-digit or four-operand addition, whereas every graph run beats it on every family at step 100 and  $\lambda = 0.1$  nearly doubles its OOD peak. Second, every run first falls below the untrained student and then recovers, and the dip is monotone

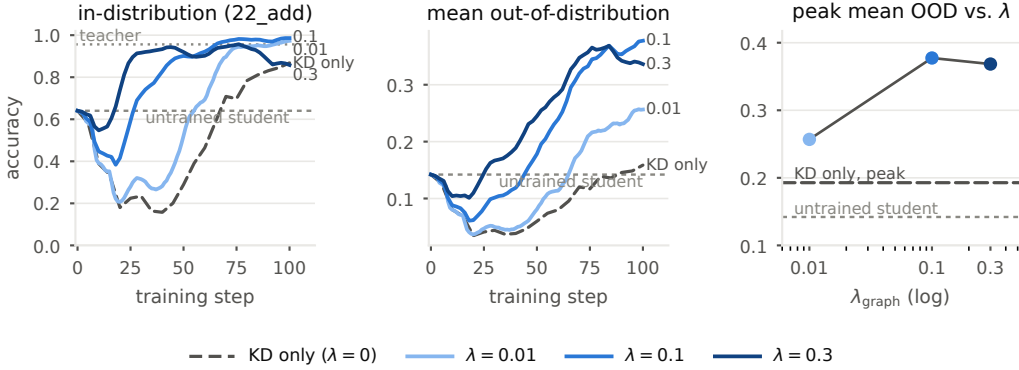


Figure 5: Sensitivity to  $\lambda$ . One hue stepped light to dark with increasing  $\lambda$ ; standard KD in dashed grey, shown to step 100 to share the axis. *Left, centre*: in-distribution and mean out-of-distribution accuracy against training step; the dashed rule is the untrained student and the dotted rule the teacher. *Right*: peak mean out-of-distribution accuracy against  $\lambda$  on a log axis, with the control’s 300-step peak as the dashed reference.

in  $\lambda$  — a minimum of 0.180, 0.207, 0.384 and 0.547 at  $\lambda = 0, 0.01, 0.1$  and  $0.3$ , recovered after 70, 55, 28 and 18 steps — so the dip belongs to the KD objective and the graph term shortens it. Third, larger  $\lambda$  rises faster but peaks earlier and then declines:  $\lambda = 0.3$  peaks between steps 76 and 84 and gives back 0.03 OOD by step 100, while  $\lambda = 0.1$  was still improving when it ended. The response is therefore an inverted U whose top is broad relative to a single seed —  $\lambda = 0.1$  and  $0.3$  differ by 0.009 OOD at their peaks across a threefold change in  $\lambda$  — and we use  $\lambda = 0.1$  throughout.

The sweep doubles as a precision check on the training stack. At step one the runs share weights, so any scale-invariant summary of the graph gradient must agree across  $\lambda$ ; both  $\|g_{\text{graph}}\|/\lambda$  and  $\cos(g_{\text{graph}}, g_{\text{KD}})$  do, to four decimals (2.82 and  $-0.041$ ). Under the bf16 gradient accumulation of an earlier version of the stack (Appendix B) they did not: the graph gradient, which arrives one prompt at a time into a buffer already holding the far larger KD gradient, was rounded away more severely the smaller  $\lambda$  was, which biased that regime against small  $\lambda$ . The  $\lambda = 1$  results of Appendix C were measured there and are being re-run. Single seed;  $\lambda = 1$  is not included here.

## E PERFORMANCE OPTIMISATIONS

An attribution graph costs one batch-expanded forward pass plus a vector–Jacobian product per node, which makes  $\mathcal{L}_{\text{graph}}$  far more expensive per prompt than  $\mathcal{L}_{\text{KD}}$ . Four optimisations bring it inside a training loop. *Probe-set caching*: the residual-stream input to every MLP over all  $10^4$  prompts is computed once per model and cached to disk, keyed by model and dataset, then read back layer-by-layer, so every activation grid is recovered from the cache rather than by re-running the model. *Analytic pre-selection*: write-vector norms factor as  $\|v_i^{\text{out}}\|_2 = |a_i| \|W_{\text{down}}[:, n_i]\|_2$ , so the top- $k$  runs in  $O(n_{\text{pos}} d_{\text{mlp}})$  space without materialising the  $[n_{\text{pos}}, d_{\text{mlp}}, d]$  tensor of write vectors. *Batching*: one batch-expanded forward pass serves a whole batch of target nodes, and all masks and activation grids are scored against each other in a single matmul for (4). *Bounded memory*: only  $n_{\text{graph}}$  prompts per batch get a graph, and each prompt’s graph loss is backpropagated and freed before the next graph is built, so peak memory is set by one prompt’s attribution graph rather than the batch’s.

## F ADDITIONAL DETAILS

[Appendix. Full hyperparameters; the complete ANOVA category list including the composite and combination-sum categories omitted from Section 3.3; additional supergraph figures; per-supernode activation-heatmap montages.]